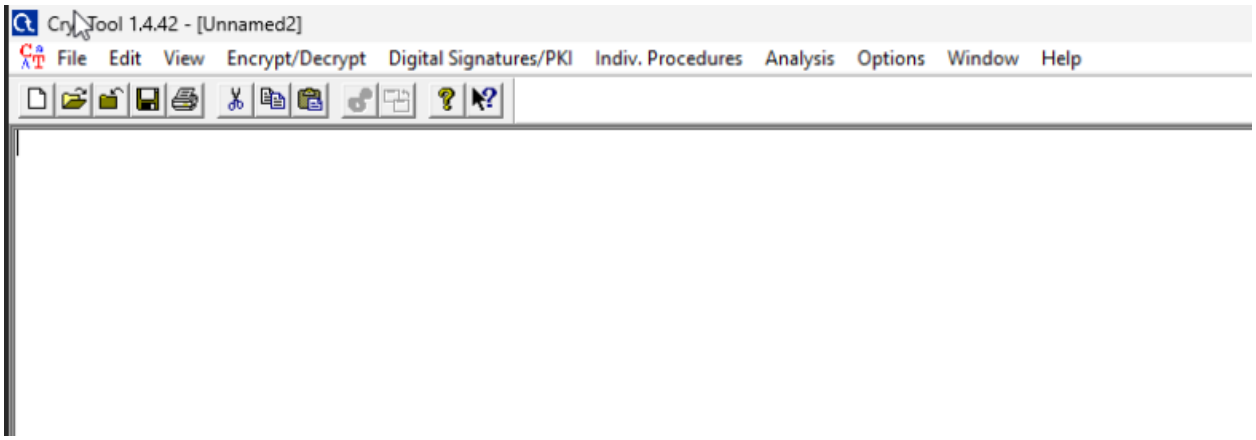
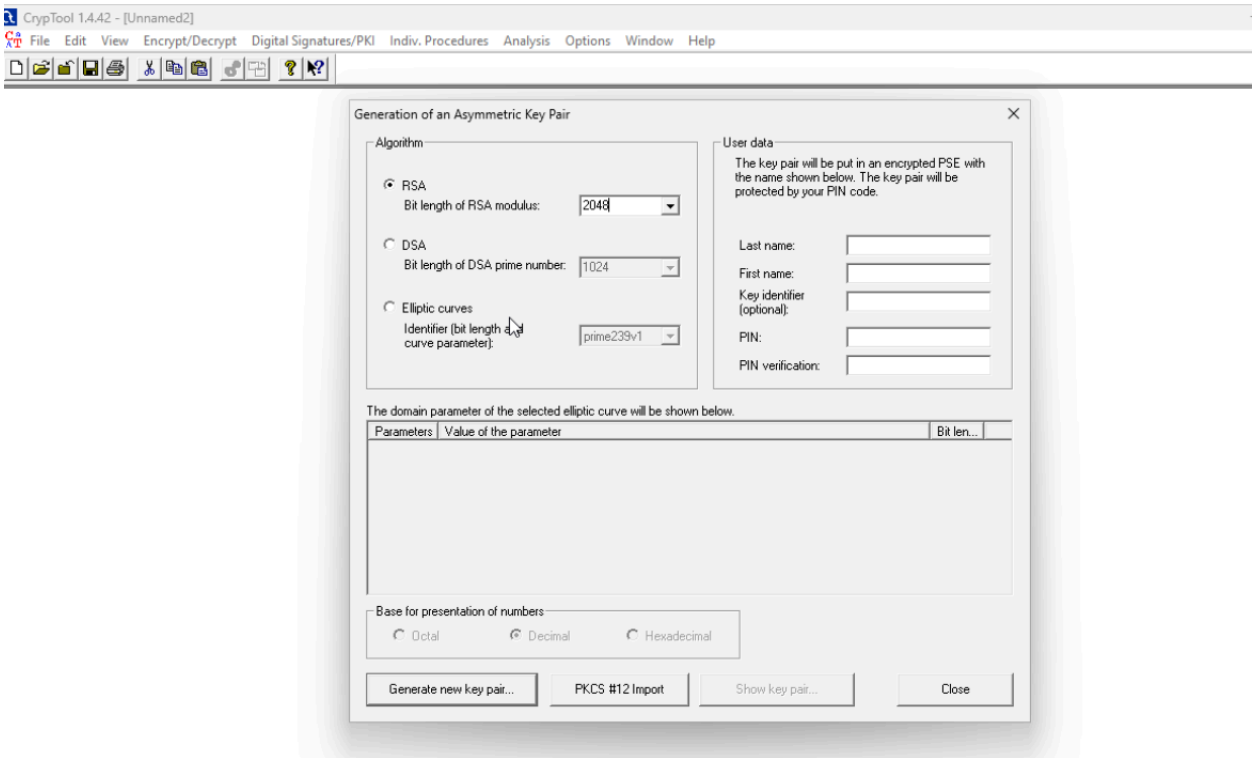
Lab: Asymmetric Encryption using CrypTool 1
Jack Irvin

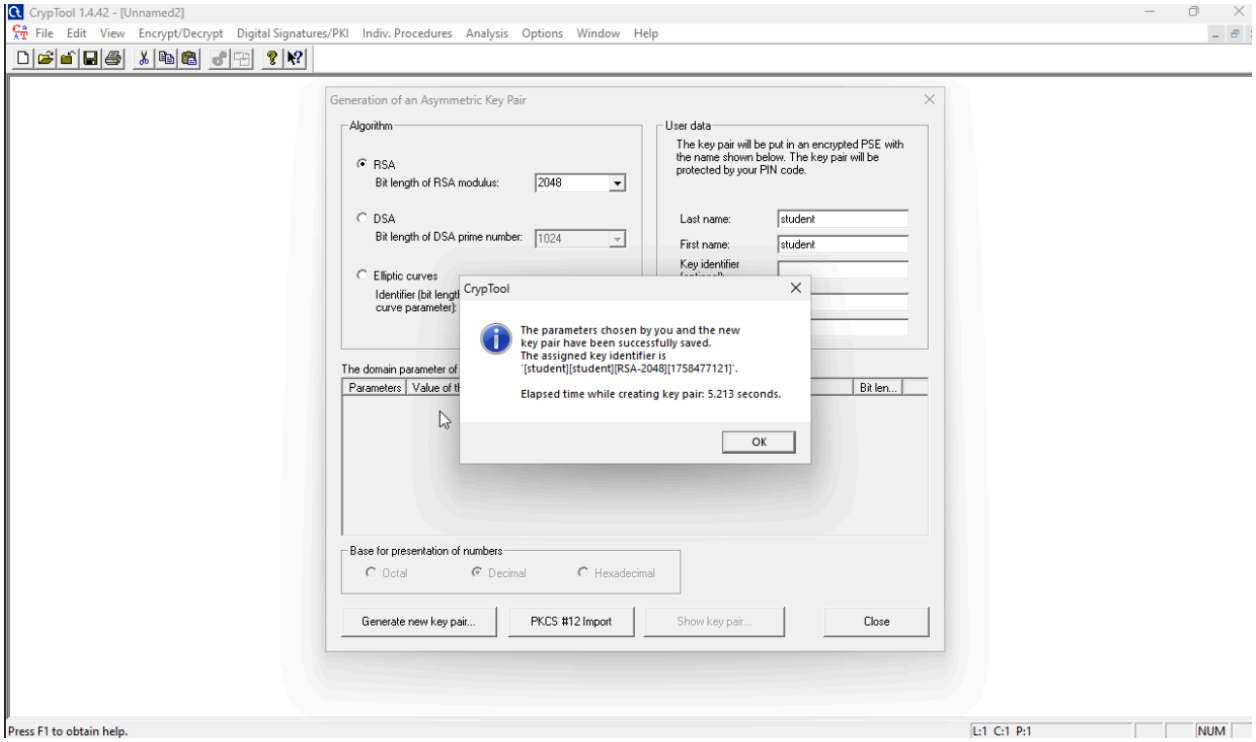
Task 1: Generate Asymmetric Key Pairs

Step 1: Open CrypTool 1



Step 2: Generate a Key Pair





Think & Answer: What happens if you choose a smaller key size (e.g., 512 bits)?

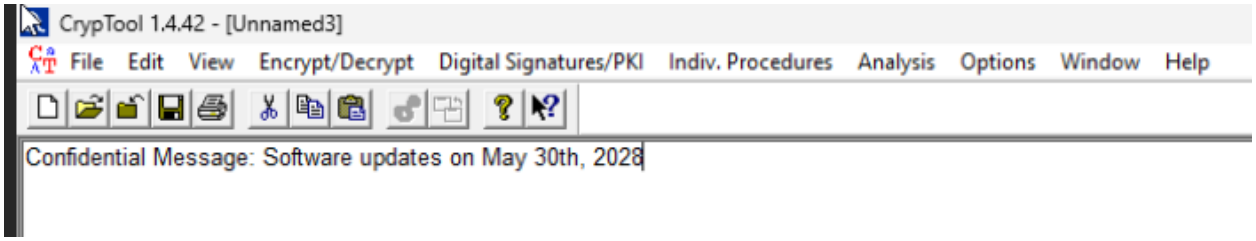
- If you choose a smaller key size your security is compromised, it makes it easier to get brute forced.

Think & Answer: Why do we need both public and private keys?

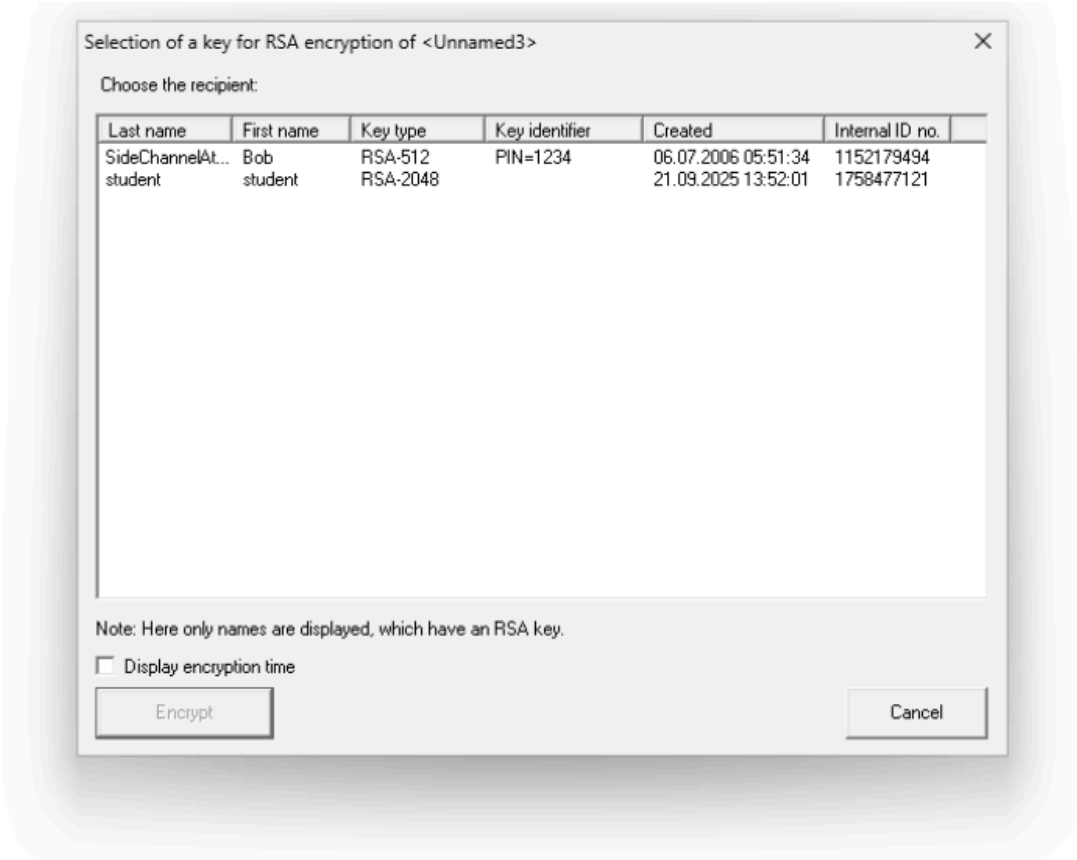
- We need public and private keys to allow secure encryption and decryption of data being transmitted.

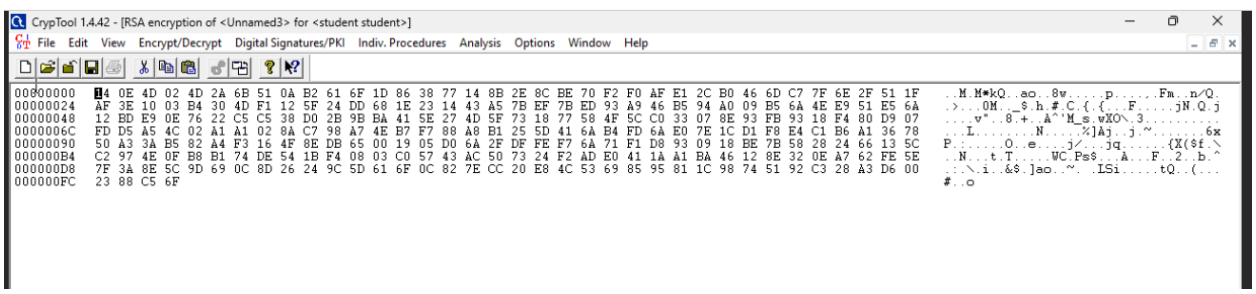
Task 2: Encrypt a Message using the Public Key

Step 3: Enter the Message



Step 4: Encrypt Using RSA





Think & Answer: What part of the encryption process ensures that only the intended recipient can decrypt it?

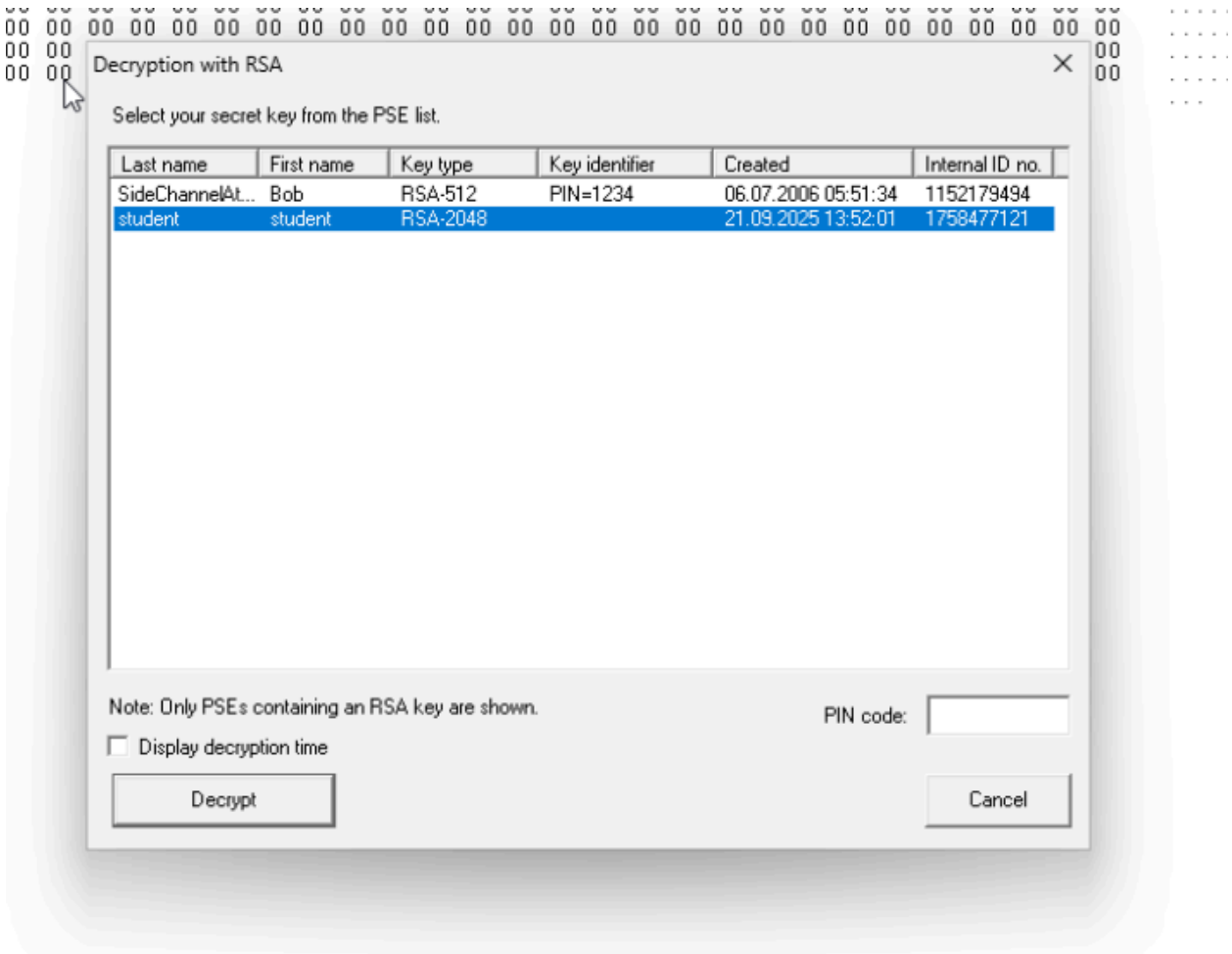
- Only the people with the keys are able to decrypt the message no matter the encryption making it secure. The only two people who will know the contents of the message and be able to translate it are the sender and receiver.

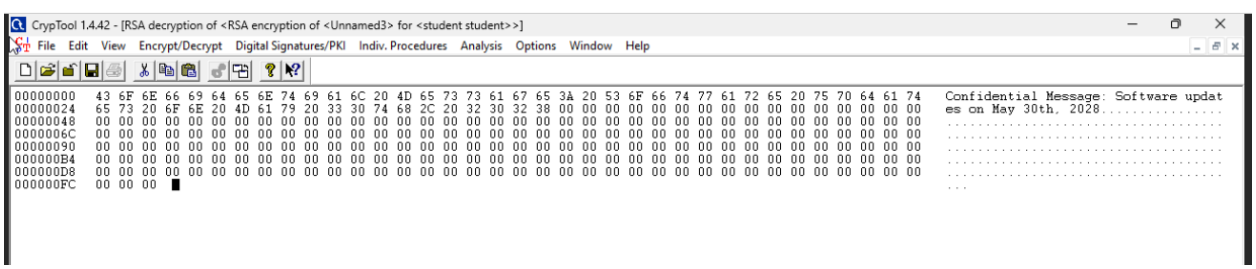
Think & Answer: If an attacker captures the encrypted message, what can they do with it?

- If an attacker captures the encrypted message they could attempt to brute force it or attempt to study the message to try to decode it.

Task 3: Decrypt the Message using the Private Key

Step 5: Decrypt the Encrypted Message





Think & Answer: What would happen if someone tried to decrypt using the public key instead of the private key?

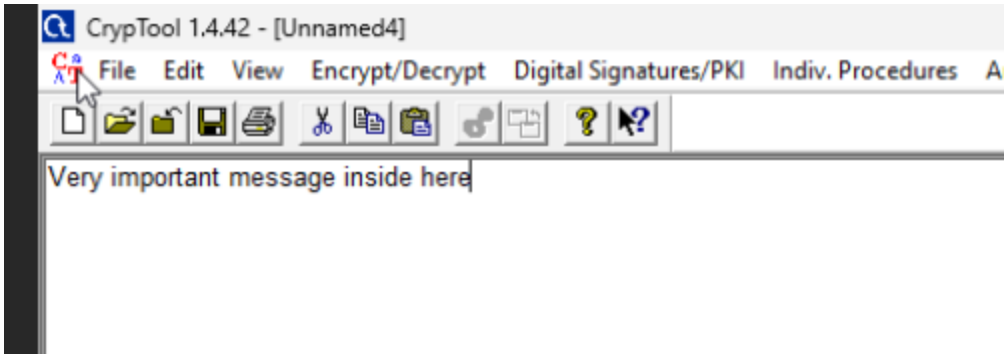
- If somebody tries to decrypt the data using the public key it will just give a mess of data. The public key is like a push and the private key is like a pull, with the data being a door. Once the door is pushed closed, you can’t just push to open it again.

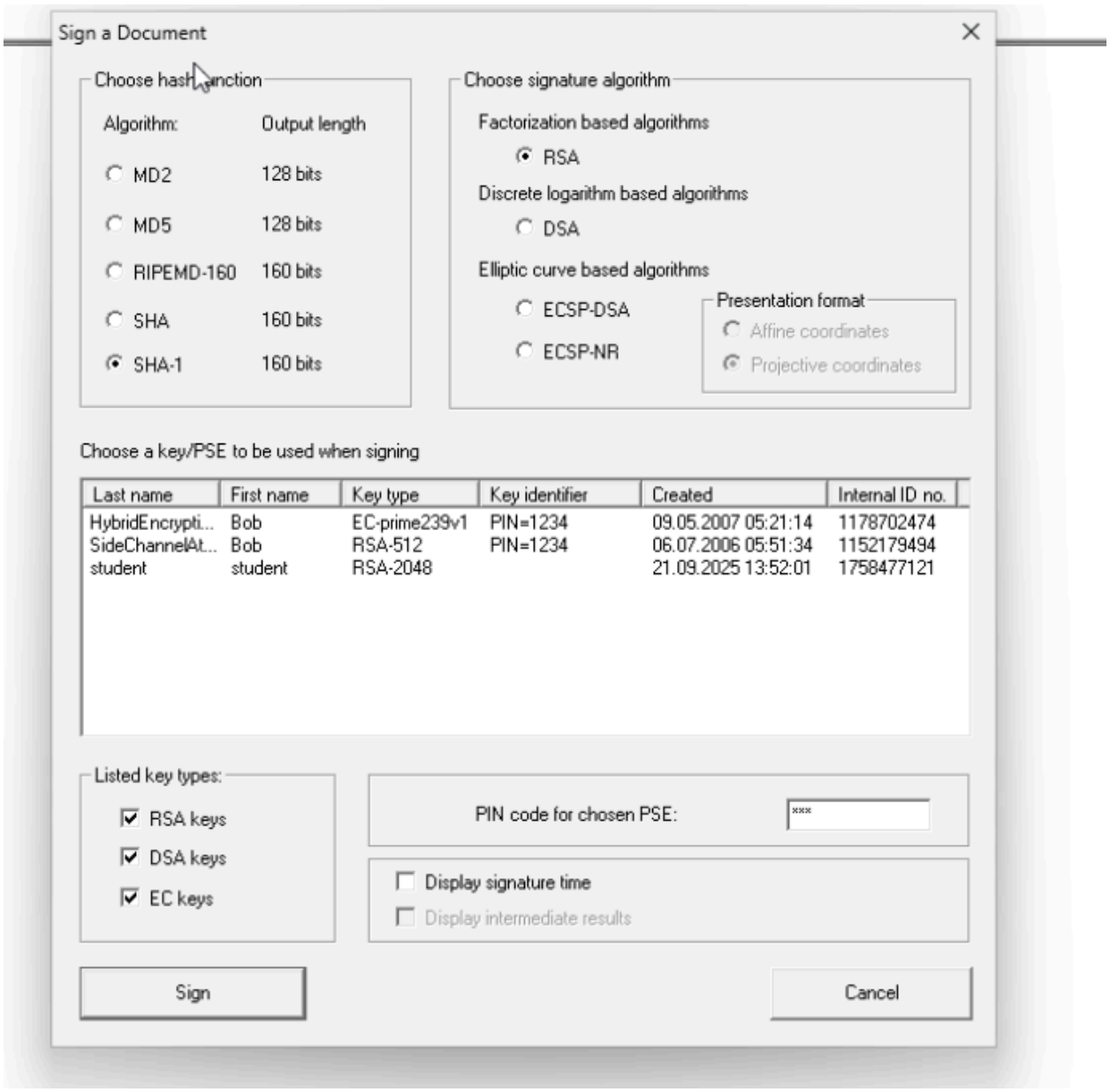
Think & Answer: Why is the private key never shared?

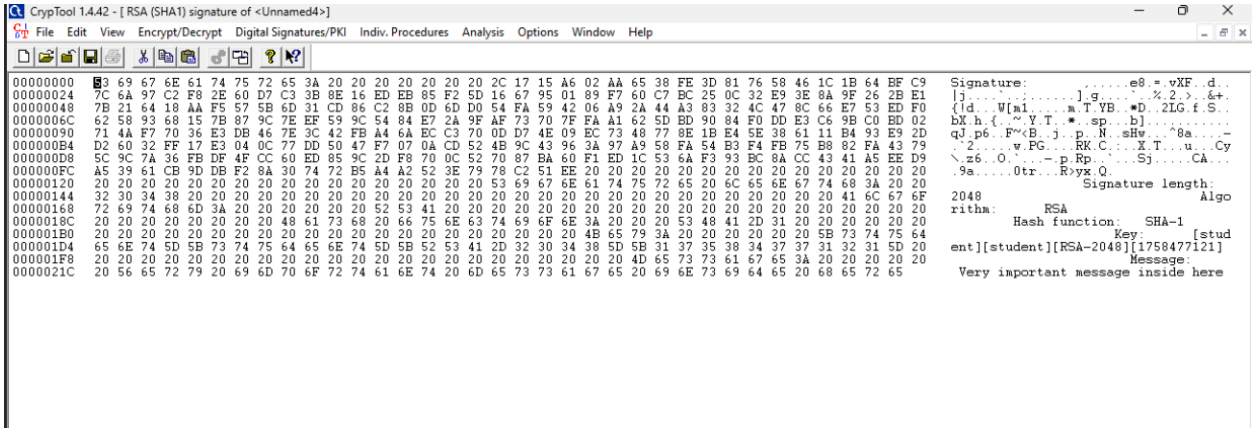
- The private key is never shared because it is the sole way to decrypt data that is asymmetrically encoded which makes it very valuable.

Task 4: Digital Signature Verification

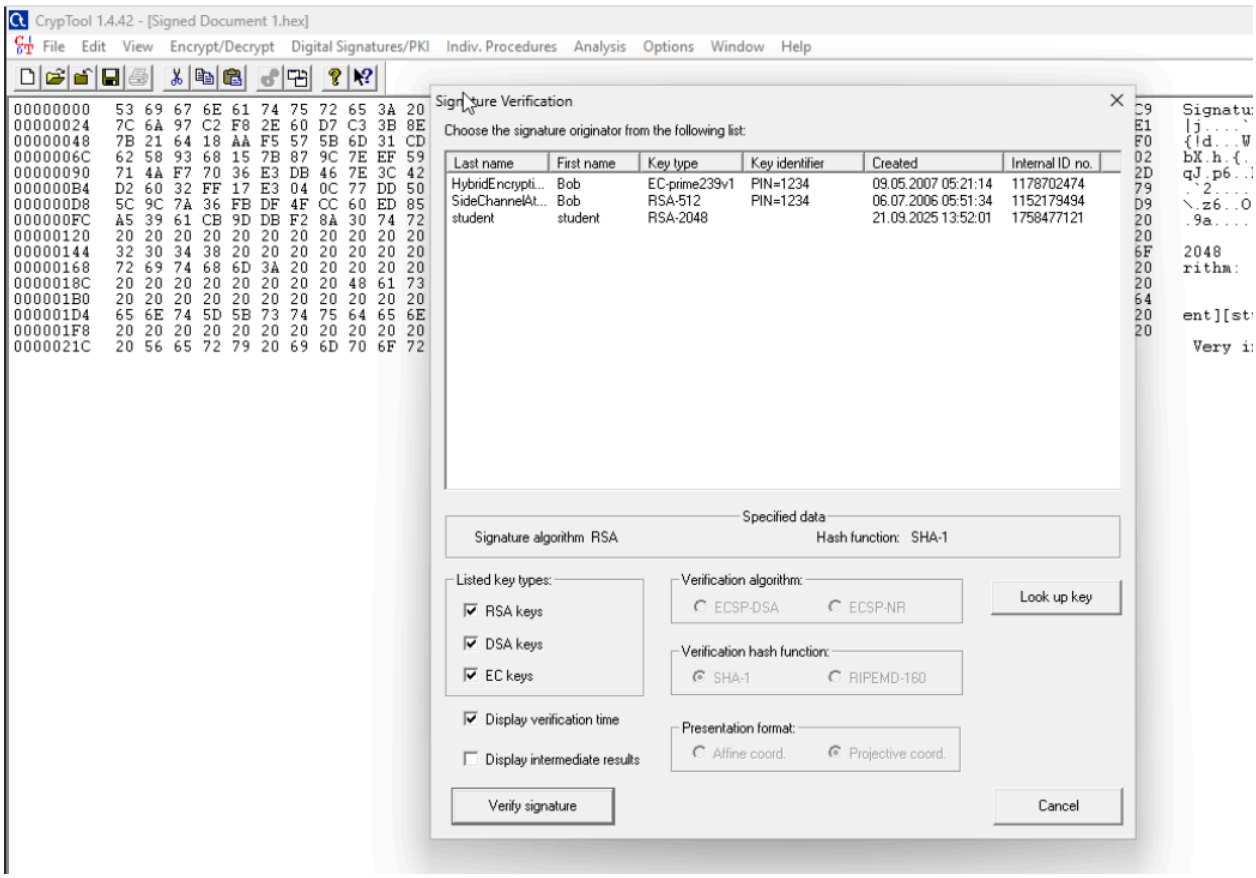
Step 6: Sign the Message

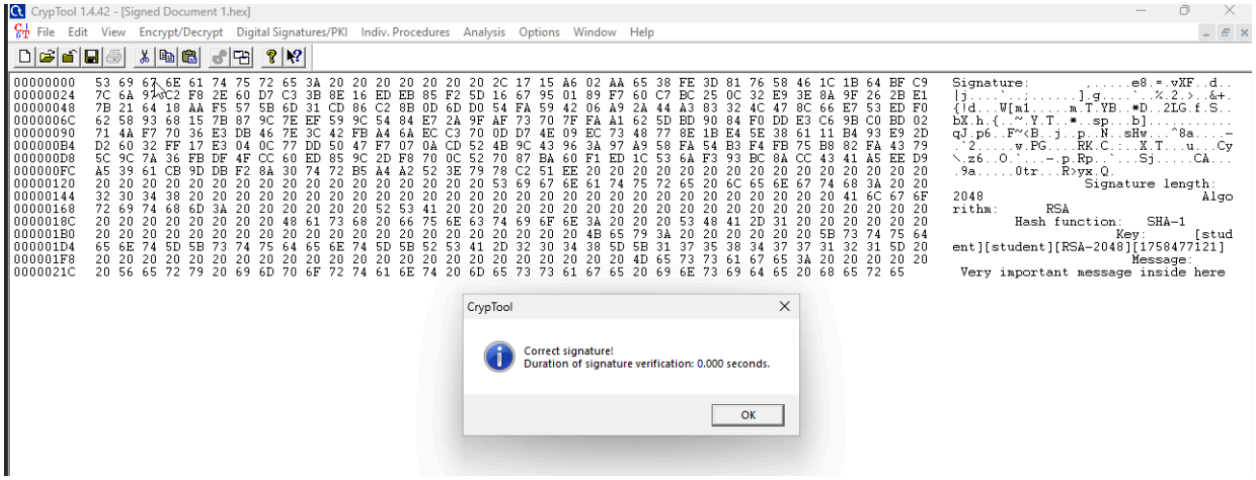

CrypTool 1.4.42 - [Unnamed4]
File Edit View Encrypt/Decrypt Digital Signatures/PKI Indiv. Procedures A
Very important message inside here


Sign a Document
Choose hash function
Algorithm: Output length
MD2 128 bits
MD5 128 bits
RIPEMD-160 160 bits
SHA 160 bits
SHA-1 160 bits
Choose signature algorithm
Factorization based algorithms
RSA
Discrete logarithm based algorithms
DSA
Elliptic curve based algorithms
ECSP-DSA
ECSP-NR
Presentation format
Affine coordinates
Projective coordinates
Choose a key/PSE to be used when signing
Last name First name Key type Key identifier Created Internal ID no.
HybridEncrypti... Bob EC-prime239v1 PIN=1234 09.05.2007 05:21:14 1178702474
SideChannelAt... Bob RSA-512 PIN=1234 06.07.2006 05:51:34 1152179494
student student RSA-2048 21.09.2025 13:52:01 1758477121
Listed key types:
RSA keys
DSA keys
EC keys
PIN code for chosen PSE:
***
Display signature time
Display intermediate results
Sign
Cancel



Step 7: Verify the Signature





Think & Answer: What would happen if an attacker modified the message after it was signed?

- If an attacker modified the message after it was signed the digital signature wouldn't match the message so the signature verification wouldn't work.

Think & Answer: Why is it important to verify a digital signature?

- It is important to verify digital signatures to tell if they were tampered with or an attacker modified the message while it was in transit.